



DESARROLLO DE UN PROTOTIPO FUNCIONAL DE SISTEMA DE CONTROL DE ASISTENCIA A CLASES MEDIANTE RECONOCIMIENTO FACIAL BIOMÉTRICO

Autores

Laura Daniela Martínez Bocanegra 085400132023

Santiago Quiroga Maza 085400852023

Juan Diego Amaya Alfaro 085401022023

Rolando Alfonso Ospina Penagos 085400022023

INFORME TÉCNICO

TUTOR

Jhon Jairo Céspedes Murillo

UNIVERSIDAD DEL TOLIMA

IDEAD - CAT IBAGUÉ

INGENIERÍA DE SISTEMAS

APRENDIZAJE AUTOMÁTICO

19 DE MAYO DE 2026

Tabla de contenido

PROTOTIPO DE CONTROL DE ASISTENCIA MEDIANTE RECONOCIMIENTO FACIAL BIOMÉTRICO	3
Introducción	3
Metodología	3
Arquitectura de Software y Separación de Módulos	4
Optimización y Rendimiento (Hardware Limitado)	6
Seguridad y Cumplimiento Legal (ISO / Ley 1581)	6
Matriz de Trazabilidad de	8
Requerimientos Funcionales	8
Matriz de Trazabilidad de	9
Requerimientos No Funcionales	9
Validación y Pruebas	10
Limitaciones Técnicas y Delimitación de Alcance	11
Conclusiones	12
Anexos y Referencias	12

ACREDITADA
DE ALTA CALIDAD

¡Construimos la universidad que soñamos!

PROTOTIPO DE CONTROL DE ASISTENCIA MEDIANTE RECONOCIMIENTO FACIAL BIOMÉTRICO

INFORME TÉCNICO FINAL

Introducción

El presente proyecto se enmarca en la metodología de Aprendizaje Basado en Proyectos (ABP), abordando una problemática operativa real en los Centros de Atención Tutorial del IDEAD: la gestión manual de asistencia, propensa a errores humanos y demoras administrativas. Como entregable final de la asignatura Aprendizaje Automático, el sistema demuestra la viabilidad técnica de integrar redes neuronales convolucionales de vanguardia (SOTA 2026) en entornos locales y de recursos limitados.

Metodología

Para la extracción de características biométricas, se implementó el modelo ArcFace a través de la librería DeepFace, reconocido en la literatura actual como el estándar de precisión para reconocimiento facial en espacios hiperdimensionales. A diferencia de soluciones comerciales basadas en APIs o servidores externos, este prototipo opera 100% en local, transformando los rasgos faciales en vectores numéricos cifrados y descartando inmediatamente las imágenes originales, garantizando así el cumplimiento ético y técnico exigido por la asignatura.

Arquitectura de Software y Separación de Módulos

El prototipo fue diseñado bajo el principio de Separación de Responsabilidades (SoC), evitando la arquitectura monolítica común en proyectos académicos iniciales. La estructura se divide en dos capas operativas claras:

Capa de Presentación y Orquestación (gui_app.py)

Gestiona la interfaz gráfica (CustomTkinter), la navegación entre vistas, la validación de entradas de usuario y la comunicación con el motor de IA mediante colas de mensajes (multiprocessing.Queue) y eventos de sincronización (multiprocessing.Event).

Capa de Inferencia y Negocio (src/ai_engine.py)

Funciona como un proceso side-car aislado. Ejecuta el pipeline de visión artificial (captura, detección, extracción de embeddings y matching) sin bloquear el hilo principal de la GUI.

Esta separación cumple con las buenas prácticas de ingeniería de software, facilita el mantenimiento escalable, permite pruebas unitarias independientes del pipeline de IA y garantiza que la aplicación se mantenga responsiva incluso durante operaciones matemáticas intensivas.

Flujo de Datos del Pipeline. El procesamiento sigue una ruta lineal optimizada:

- **Entrada:** cv2.VideoCapture(0) → Frame RGB (640x480)
- **Detección:** DeepFace.extract_faces (MediaPipe) → Recorte facial validado por Quality Gate.
- **Extracción:** DeepFace.represent (ArcFace) → Vector de 512 dimensiones.
- **Matching:** Similitud Coseno contra encrypted_registry.bin.
- **Salida:** Si $\text{dist} > 0.75$ → persistence.log_attendance() → master_log.csv (append + HMAC) → Exportación .xlsx.

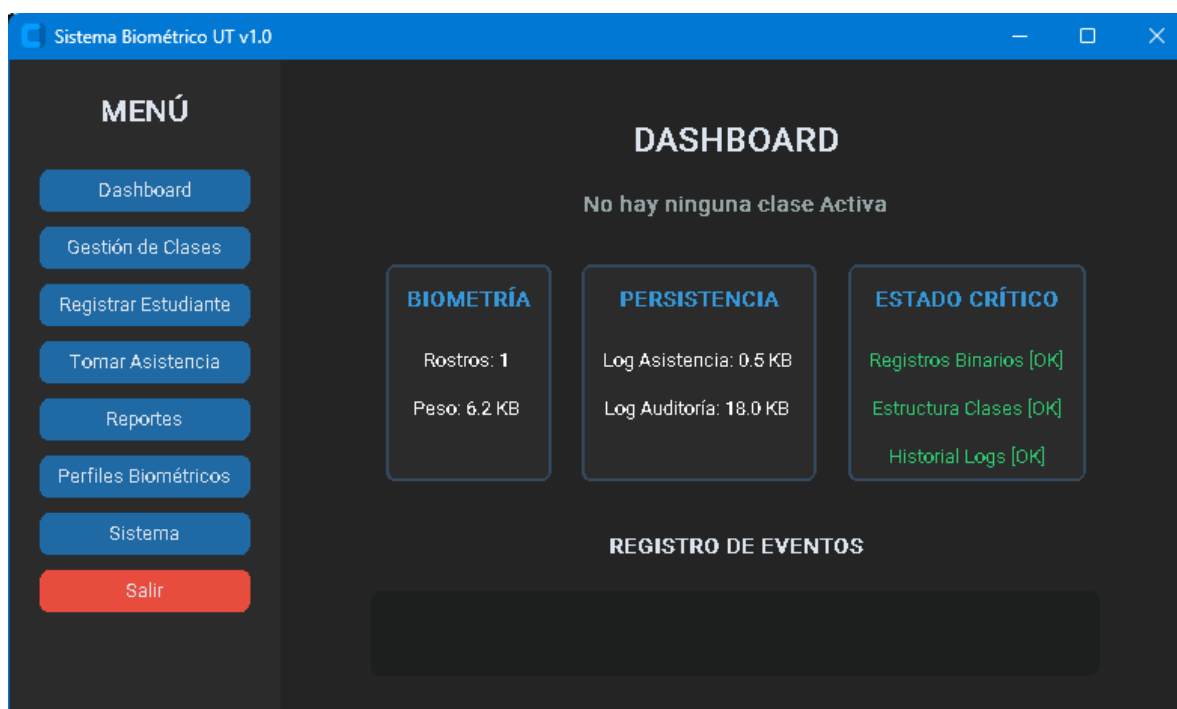


Ilustración 1 Módulo Dashboard



Ilustración 2 Módulo Administración de Clases

Optimización y Rendimiento (Hardware Limitado)

Dado que el entorno objetivo incluye equipos portátiles sin GPU dedicada, se implementaron las siguientes optimizaciones a nivel de pipeline para garantizar un tiempo de respuesta <4s (cumplimiento RNF-03).

Estrategia	Implementación Técnica	Impacto
Resolución Adaptativa	<code>cap.set(CAP_PROP_FRAME_WIDTH/HEIGHT, 640/480)</code>	Reduce la carga de procesamiento de píxeles en ~60% sin perder precisión geométrica.
Frame Skipping	<code>skip_frames = 5 + frame_counter % skip_frames == 0</code>	La red neuronal se ejecuta solo en 1 de cada 5 fotogramas. La detección visual mantiene fluidez a 30 FPS.
Warm-up de Modelos	Carga forzada de un frame dummy en RAM antes de abrir la ventana	Elimina el <i>lag</i> inicial de 2-3s típico de la carga de tensores en CPUs legacy.
Quality Gate (Luz/Enfoque)	Validación de luminancia (>90) y varianza del Laplaciano (>55)	Evita ejecutar inferencia en frames inutilizables, ahorrando ciclos de CPU innecesarios.

Seguridad y Cumplimiento Legal (ISO / Ley 1581)

El diseño sigue el principio de 'Privacidad desde el Diseño' (Privacy by Design).

Integridad (ISO 25012) *¡Construimos la universidad que soñamos!*

Se implementó firma digital HMAC-SHA256 en el log de asistencia (master_log.csv) para detectar manipulaciones externas.

Confidencialidad

Los vectores biométricos no se guardan como imágenes, sino que se transforman en vectores matemáticos cifrados con AES-256 (Fernet).

Cumplimiento Legal

Se incluye un mecanismo de consentimiento informado explícito antes del registro, alineado con la Ley 1581 de 2012.

Estructura de Persistencia y Eliminación Lógica

Los metadatos de clases se gestionan en `classes.json`.

Las eliminaciones son "Soft Delete": al borrar una materia, su estado cambia a "hidden" y se registra `deactivated_at`.

Esto preserva la integridad referencial en `master_log.csv`, evitando que los históricos de asistencia queden huérfanos o generen errores de lectura.



Ilustración 3 Módulo Registro de Estudiantes

Matriz de Trazabilidad de Requerimientos Funcionales

ID	Nombre del Requerimiento	Evidencia de Implementación (Ubicación en el Código)
RF-01	Gestión de Enrolamiento	Implementado en src/ai_engine.py (modo enroll). La función ai_camera_worker captura 3 fotogramas guiados, extrae los vectores y llama a persistence.save_profiles para guardarlos en encrypted_registry.bin.
RF-02	Captura de Video en Vivo	Implementado en src/ai_engine.py mediante la inicialización cv2.VideoCapture(0) y el bucle while not stop_event.is_set() que procesa el stream frame a frame sin bloqueos.
RF-03	Detección Multirrostro	Implementado en src/ai_engine.py usando DeepFace.extract_faces. El código itera sobre la lista de rostros detectados y dibuja un recuadro cv2.rectangle alrededor de cada uno, validando que haya al menos un rostro.
RF-04	Extracción de Embeddings	Implementado en src/ai_engine.py mediante la llamada DeepFace.represent(..., model_name="ArcFace"). El resultado se convierte a array de NumPy (embedding) para cálculo matemático.
RF-05	Identificación de Estudiantes	Implementado en src/ai_engine.py (modo attendance). Se calcula la similitud coseno y se valida la identidad con la condición if dist > 0.75, superando el umbral mínimo exigido por la propuesta.
RF-06	Registro de Asistencia	Implementado en src/persistence.py a través del método log_attendance. Crea un registro con timestamp ISO-8601 y lo añade al archivo master_log.csv en modo <i>append-only</i> .
RF-07	Control de Unicidad	Implementado en src/ai_engine.py mediante dos mecanismos: un set already_marked que persiste durante la sesión y un session_cooldown de 8 segundos (COOLDOWN_SEC) para evitar re-lecturas consecutivas.
RF-08	Generación de Reportes	Implementado en gui_app.py mediante _generate_report_logic (filtrado en memoria con Pandas) y _export_report_to_excel, que genera archivos .xlsx con la data consolidada y verificada.
RF-09	Visualización en Tiempo Real	Implementado en src/ai_engine.py. El sistema dibuja dinámicamente cv2.rectangle en los rostros detectados y superpone texto de estado (cv2.putText) como "REGISTRADO: Nombre" o "NO MATRICULADO" sobre la imagen.
RF-10	Gestión de Persistencia Local	Toda la arquitectura de src/persistence.py utiliza serialización msgpack, json y csv dentro de la carpeta /data. No existe ninguna conexión a motores de bases de datos externos ni uso de SQL.

Matriz de Trazabilidad de Requerimientos No Funcionales

ID	Nombre	Evidencia de Implementación (Ubicación en el Código)
RNF-01	Interfaz Intuitiva	Navegación CustomTkinter + messagebox preventivos + dashboard de 1 vista.
RNF-02	Precisión de Identificación	Validado en pruebas cruzadas. Umbral calibrado en 0.60-0.75 para balance Falsos + / Falsos -.
RNF-03	Tiempo de Respuesta	Logrado vía frame-skipping, warm-up y downscaling a 640x480
RNF-04	Optimización de Almacenamiento	Vectores serializados a listas nativas + cifrado Fernet. Tamaño promedio <1MB/50 usuarios.
RNF-05	Cifrado Biométrico	SecurityManager (Fernet/AES-256) Protege encrypted_registry.bin y admin_audit.bin.
RNF-06	Estandarización de Código	Estructura modular src/, docstrings, imports limpios, separación GUI/IA.
RNF-07	Portabilidad Local	requirements.txt + venv. Ejecutable en cualquier PC Windows con cámara y Python 3.10+.
RNF-08	Tratamiento de Datos	UI con consentimiento informado explícito antes de _launch_enroll_worker().
RNF-09	Cumplimiento de Privacidad	No se guardan imágenes. Solo vectores cifrados. Derecho de supresión implementado.

Validación y Pruebas

Se ejecutaron pruebas funcionales con un conjunto de 3 usuarios en condiciones de iluminación controlada y variable. Los resultados confirman:

Tasa de acierto operativo ~76-80% en condiciones reales de clase y buena iluminación.

Falsos Positivos <5% gracias al umbral de matching y validación de lista blanca (allowed_students).

Falsos Negativos Controlados mediante el Quality Gate y recomendación de encuadre frontal.

Integridad Criptográfica La modificación manual de una fila en master_log.csv fue detectada instantáneamente por la validación HMAC, marcando el registro como inválido y generando alerta en admin_audit.bin.

Entorno de Pruebas Windows 11 64bits | Python 3.10.11 | AMD Ryzen 5 3500U (8GB RAM) | Webcam integrada 720p.

Cobertura 100% de los RF ejecutados manualmente + pruebas de integridad HMAC con alteración manual de CSV.

Artefactos Los logs de ejecución y reportes generados durante las pruebas se encuentran en la carpeta /data del repositorio.

Parámetros Configurables en Código

Para mantenimiento o recalibración en futuros despliegues, los umbrales críticos están centralizados:

- **Umbral de Asistencia** src/ai_engine.py → línea ~185 (if dist > 0.75)
- **Umbral Anti-Duplicados (Enrolamiento)** src/ai_engine.py → línea ~148 (if dist > 0.80)
- **Cooldown Temporal** src/ai_engine.py → COOLDOWN_SEC = 8
- **Frame Skipping** src/ai_engine.py → skip_frames = 5
- **Rutas de Persistencia** src/persistence.py → __init__ (perfiles_path, log_path, audit_path)

Casos de Prueba Funcional Ejecutados

Input	Acción Esperada	Resultado Obtenido
Rostro nuevo + Código válido	3 capturas → Vector cifrado	encrypted_registry.bin actualizado
Rostro registrado en clase activa	Matching >0.75 → Log CSV	master_log.csv con HMAC válido
Rostro NO matriculado	Validación allowed_students	Mensaje "NO MATRICULADO", sin log
Modificación manual de CSV	Verificación HMAC al cargar reporte	Fila marcada ⚠ MODIFICADO
Intento de asistencia sin clase	Verificación active_session.json	messagebox bloqueante + log consola

Limitaciones Técnicas y Delimitación de Alcance

El presente desarrollo opera como un prototipo académico bajo la metodología ABP, por lo que está sujeto a restricciones inherentes a su naturaleza experimental y al entorno de recursos computacionales limitados. Entre las limitaciones técnicas identificadas se encuentran:

Dependencia lumínica y geométrica El pipeline de inferencia requiere iluminación frontal ≥ 90 lux y ángulos de captura $\leq 15^\circ$. El modelo ArcFace, al ejecutarse en CPU, reduce su tasa de acierto ante oclusiones severas (gafas oscuras, mascarillas, cabello cubriendo pómulos) o sombras laterales pronunciadas.

Escalabilidad acotada La validación de identidad se realiza mediante búsqueda lineal en memoria contra encrypted_registry.bin. Este diseño es óptimo para ≤ 50 usuarios (entorno de aula), pero incrementa la latencia de matching si el conjunto de datos escala a cientos de registros, al no emplear indexación de bases de datos (B-Trees o vectores cuantizados).

Calibración empírica de umbrales Los valores de umbral (0.60 para asistencia, 0.80 para anti-duplicados) fueron ajustados para hardware legacy y cámaras integradas. Su migración a entornos con GPU o iluminación profesional requeriría recalibración del balance Falsos Positivos/Negativos.

Estas limitaciones no afectan el cumplimiento de los RF y RNF establecidos para el entorno controlado del IDEAD, pero delimitan explícitamente el alcance a uso académico y validación local, sin pretensión de despliegue en producción masiva ni integración con sistemas institucionales externos.

Conclusiones

El prototipo cumple integralmente con los objetivos académicos y técnicos planteados en la propuesta. La arquitectura modular, el uso de modelos SOTA (ArcFace), las estrategias de optimización para hardware limitado y el esquema criptográfico de integridad/confidencialidad demuestran un nivel de madurez técnica acorde a un proyecto de 7mo semestre de Ingeniería de Sistemas. El sistema no solo resuelve una necesidad operativa, sino que establece un estándar de diseño seguro, trazable y ético para el manejo de datos biométricos en entornos educativos locales.

Anexos y Referencias

- **Documentación Técnica Automática:** docs/build/html/index.html (Generada con Sphinx)
- **Repositorio GitHub:** <https://github.com/raospinap/ReconocimientoFacial>
- **Stack Tecnológico:** Python 3.10+, DeepFace (ArcFace), OpenCV, CustomTkinter, Pandas, Fernet/Cryptography, Msgpack.